

Implementación de Base de Datos - ERP JD Refrigeración S.A.C.

Este documento detalla la implementación del modelo relacional en Oracle, cubriendo las **15 tablas** del ERP organizadas en 9 módulos de negocio. Los nombres de tablas, columnas y enums corresponden exactamente a las entidades JPA del backend Spring Boot.

Participantes del Proyecto

- [Apaza Mamani Wilson Ronald]
 - [Magaño Quispe Aron Rodrigo]
 - [Chambi Sirena Yoel Wagner]
-

Modelo Relacional - Resumen de Tablas

#	Tabla Oracle	Módulo	Tipo
1	CATEGORIAS	Inventario	Catálogo
2	PRODUCTOS	Inventario	Catálogo
3	CLIENTES	Clientes	Catálogo
4	PROVEEDORES	Proveedores	Catálogo
5	COMPRAS	Compras	Transaccional
6	DETALLE_COMPRA	Compras	Transaccional
7	VENTAS	Ventas	Transaccional
8	DETALLE_VENTAS	Ventas	Transaccional
9	PAGOS	Ventas	Transaccional
10	COTIZACIONES	Cotizaciones	Transaccional
11	DETALLE_COTIZACIONES	Cotizaciones	Transaccional
12	COMPROBANTES_ELECTRONICOS	Facturación	Transaccional
13	INSTALACIONES	Postventa	Transaccional
14	MANTENIMIENTOS	Postventa	Transaccional
15	NOTIFICACIONES	Notificaciones	Transversal

1. Creación de Tablas (DDL)

Las tablas se crean en orden de dependencias: primero las independientes (catálogos), luego las que tienen foreign keys.

```
-- =====
-- MÓDULO: INVENTARIO
-- =====

CREATE TABLE categorias (
    id                NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
    nombre            VARCHAR2(120) NOT NULL,
    descripcion       VARCHAR2(250),
    activo            NUMBER(1) DEFAULT 1 NOT NULL
);

CREATE TABLE productos (
    id                NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
    nombre            VARCHAR2(150) NOT NULL,
    descripcion       VARCHAR2(300),
    precio            NUMBER(12,2) NOT NULL,
    stock             NUMBER(10) NOT NULL,
    stock_minimo      NUMBER(10) NOT NULL,
    activo            NUMBER(1) DEFAULT 1 NOT NULL,
    categoria_id      NUMBER(19) NOT NULL,
    CONSTRAINT fk_prod_cat FOREIGN KEY (categoria_id) REFERENCES categorias(id)
);

-- =====
-- MÓDULO: CLIENTES
-- =====

CREATE TABLE clientes (
    id                NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
    tipo_cliente      VARCHAR2(20) NOT NULL,    -- PERSONA | EMPRESA
    numero_documento  VARCHAR2(11) NOT NULL UNIQUE,
    nombres           VARCHAR2(120),
    apellidos         VARCHAR2(120),
    razon_social      VARCHAR2(200),
    nombre_comercial  VARCHAR2(220) NOT NULL,
    direccion         VARCHAR2(250),
    estado_contribuyente VARCHAR2(80),
    condicion_contribuyente VARCHAR2(80),
    correo            VARCHAR2(150),
    telefono          VARCHAR2(20),
    persona_contacto  VARCHAR2(120),
    activo            NUMBER(1) DEFAULT 1 NOT NULL
);

-- =====
-- MÓDULO: PROVEEDORES
-- =====

CREATE TABLE proveedores (
    id                NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
    tipo              VARCHAR2(20) NOT NULL,    -- PERSONA_NATURAL | EMPRESA
    nombre_o_razon_social VARCHAR2(150) NOT NULL,
```

```

        ruc                VARCHAR2(11),
        dni                VARCHAR2(8),
        direccion          VARCHAR2(200),
        telefono           VARCHAR2(20),
        correo              VARCHAR2(100),
        contacto_vendedor  VARCHAR2(150)
    );

-- =====
-- MÓDULO: COMPRAS
-- =====
CREATE TABLE compras (
    id                NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
    proveedor_id      NUMBER(19) NOT NULL,
    fecha_compra      DATE NOT NULL,
    numero_factura     VARCHAR2(50),
    moneda            VARCHAR2(10) NOT NULL,          -- PEN | USD
    total             NUMBER(12,2) NOT NULL,
    CONSTRAINT fk_compra_prov FOREIGN KEY (proveedor_id) REFERENCES
proveedores(id)
);

CREATE TABLE detalle_compra (
    id                NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
    compra_id         NUMBER(19) NOT NULL,
    producto_id       NUMBER(19) NOT NULL,
    cantidad          NUMBER(10) NOT NULL,
    precio_unitario_compra NUMBER(12,2) NOT NULL,
    CONSTRAINT fk_dc_compra FOREIGN KEY (compra_id) REFERENCES compras(id),
    CONSTRAINT fk_dc_prod FOREIGN KEY (producto_id) REFERENCES productos(id)
);

-- =====
-- MÓDULO: VENTAS
-- =====
CREATE TABLE ventas (
    id                NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
    fecha            TIMESTAMP(6) NOT NULL,
    estado           VARCHAR2(30) NOT NULL,          -- REGISTRADA | FACTURADA |
ANULADA
    moneda           VARCHAR2(10) NOT NULL,          -- PEN | USD
    tipo_cambio      NUMBER(12,4) NOT NULL,
    cliente_id       NUMBER(19) NOT NULL,
    subtotal         NUMBER(12,2) NOT NULL,
    descuento        NUMBER(12,2) NOT NULL,
    igv              NUMBER(12,2) NOT NULL,
    total            NUMBER(12,2) NOT NULL,
    total_soles      NUMBER(12,2) NOT NULL,
    total_dolares     NUMBER(12,2) NOT NULL,
    CONSTRAINT fk_venta_cli FOREIGN KEY (cliente_id) REFERENCES clientes(id)
);

CREATE TABLE detalle_ventas (
    id                NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,

```

```

venta_id          NUMBER(19) NOT NULL,
producto_id       NUMBER(19) NOT NULL,
cantidad          NUMBER(10) NOT NULL,
precio_unitario   NUMBER(12,2) NOT NULL,
subtotal          NUMBER(12,2) NOT NULL,
CONSTRAINT fk_dv_venta FOREIGN KEY (venta_id) REFERENCES ventas(id),
CONSTRAINT fk_dv_prod FOREIGN KEY (producto_id) REFERENCES productos(id)
);

CREATE TABLE pagos (
  id                NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
  fecha_pago        TIMESTAMP(6) NOT NULL,
  venta_id          NUMBER(19) NOT NULL,
  moneda            VARCHAR2(20) NOT NULL,          -- PEN | USD
  tipo_cambio       NUMBER(12,4) NOT NULL,
  monto             NUMBER(12,2) NOT NULL,
  monto_soles       NUMBER(12,2) NOT NULL,
  monto_dolares     NUMBER(12,2) NOT NULL,
  metodo_pago       VARCHAR2(30) NOT NULL,          -- EFECTIVO | TRANSFERENCIA
  | YAPE | PLIN | TARJETA
  observacion       VARCHAR2(250),
  CONSTRAINT fk_pago_venta FOREIGN KEY (venta_id) REFERENCES ventas(id)
);

-- =====
-- MÓDULO: COTIZACIONES
-- =====

CREATE TABLE cotizaciones (
  id                NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
  fecha             TIMESTAMP(6) NOT NULL,
  estado            VARCHAR2(30) NOT NULL,          -- BORRADOR | ENVIADA |
  APROBADA | RECHAZADA | CONVERTIDA
  moneda            VARCHAR2(10) NOT NULL,
  tipo_cambio       NUMBER(12,4) NOT NULL,
  cliente_id        NUMBER(19) NOT NULL,
  subtotal          NUMBER(12,2) NOT NULL,
  descuento         NUMBER(12,2) NOT NULL,
  igv               NUMBER(12,2) NOT NULL,
  total             NUMBER(12,2) NOT NULL,
  total_soles       NUMBER(12,2) NOT NULL,
  total_dolares     NUMBER(12,2) NOT NULL,
  observaciones     VARCHAR2(500),
  CONSTRAINT fk_cotiz_cli FOREIGN KEY (cliente_id) REFERENCES clientes(id)
);

CREATE TABLE detalle_cotizaciones (
  id                NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
  cotizacion_id     NUMBER(19) NOT NULL,
  producto_id       NUMBER(19) NOT NULL,
  cantidad          NUMBER(10) NOT NULL,
  precio_unitario   NUMBER(12,2) NOT NULL,
  subtotal          NUMBER(12,2) NOT NULL,
  CONSTRAINT fk_dcot_cotiz FOREIGN KEY (cotizacion_id) REFERENCES
cotizaciones(id),

```

```

        CONSTRAINT fk_dcot_prod FOREIGN KEY (producto_id) REFERENCES productos(id)
    );

-- =====
-- MÓDULO: FACTURACIÓN
-- =====
CREATE TABLE comprobantes_electronicos (
    id NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
    tipo_comprobante VARCHAR2(20) NOT NULL,          -- FACTURA | BOLETA
    serie VARCHAR2(10) NOT NULL,
    correlativo VARCHAR2(20) NOT NULL,
    fecha_emision TIMESTAMP(6) NOT NULL,
    moneda VARCHAR2(10) NOT NULL,
    tipo_cambio NUMBER(12,4) NOT NULL,
    subtotal NUMBER(12,2) NOT NULL,
    descuento NUMBER(12,2) NOT NULL,
    igv NUMBER(12,2) NOT NULL,
    total NUMBER(12,2) NOT NULL,
    total_soles NUMBER(12,2) NOT NULL,
    total_dolares NUMBER(12,2) NOT NULL,
    estado_sunat VARCHAR2(40) NOT NULL,              -- GENERADO | ENVIADO |
    ACEPTADO | RECHAZADO
    mensaje_sunat VARCHAR2(500),
    estado_envio_cliente VARCHAR2(40) NOT NULL,      -- NO_ENVIADO |
    ENVIADO_CORREO | ENVIADO_WHATSAPP
    correo_destino VARCHAR2(150),
    telefono_destino VARCHAR2(20),
    nombre_archivo_xml VARCHAR2(300),
    nombre_archivo_cdr VARCHAR2(300),
    nombre_archivo_pdf VARCHAR2(300),
    fecha_envio_sunat TIMESTAMP(6),
    fecha_respuesta_sunat TIMESTAMP(6),
    fecha_envio_cliente TIMESTAMP(6),
    venta_id NUMBER(19) NOT NULL,
    CONSTRAINT fk_comp_venta FOREIGN KEY (venta_id) REFERENCES ventas(id)
);

-- =====
-- MÓDULO: POSTVENTA
-- =====
CREATE TABLE instalaciones (
    id NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
    venta_id NUMBER(19) NOT NULL UNIQUE,
    fecha_instalacion DATE NOT NULL,
    fecha_fin_garantia DATE NOT NULL,
    fecha_proximo_mantenimiento DATE NOT NULL,
    direccion_instalacion VARCHAR2(250),
    observaciones VARCHAR2(500),
    CONSTRAINT fk_inst_venta FOREIGN KEY (venta_id) REFERENCES ventas(id)
);

CREATE TABLE mantenimientos (
    id NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
    instalacion_id NUMBER(19) NOT NULL,

```

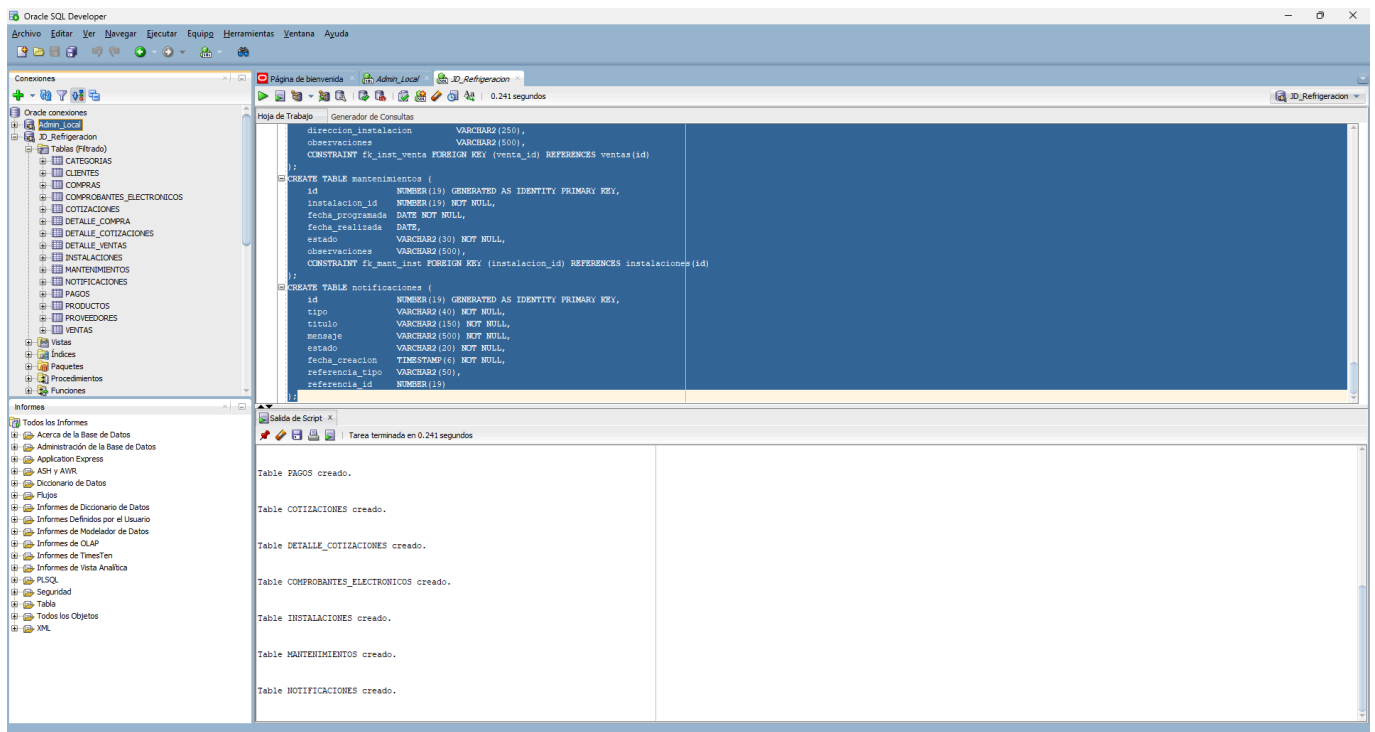
```

    fecha_programada    DATE NOT NULL,
    fecha_realizada     DATE,
    estado              VARCHAR2(30) NOT NULL,          -- PENDIENTE | CONTACTADO |
AGENDADO | REALIZADO | CANCELADO
    observaciones       VARCHAR2(500),
    CONSTRAINT fk_mant_inst FOREIGN KEY (instalacion_id) REFERENCES
instalaciones(id)
);

-- =====
-- MÓDULO: NOTIFICACIONES
-- =====

CREATE TABLE notificaciones (
    id                  NUMBER(19) GENERATED AS IDENTITY PRIMARY KEY,
    tipo               VARCHAR2(40) NOT NULL,          -- MANTENIMIENTO_PROXIMO |
GARANTIA_POR_VENCER | STOCK_BAJO
    titulo             VARCHAR2(150) NOT NULL,
    mensaje            VARCHAR2(500) NOT NULL,
    estado             VARCHAR2(20) NOT NULL,          -- NO_LEIDA | LEIDA
    fecha_creacion     TIMESTAMP(6) NOT NULL,
    referencia_tipo     VARCHAR2(50),                  -- Referencia polimórfica
(ej: 'INSTALACION', 'VENTA')
    referencia_id       NUMBER(19)                    -- ID del registro
referenciado
);

```



2. Población de Datos (INSERT)

Los datos se insertan respetando el orden de dependencias (primero catálogos, luego transaccionales).

```

-- =====
-- MÓDULO 1: INVENTARIO (Categorías y Productos)
-- =====
INSERT INTO categorias (id, nombre, descripcion, activo)
VALUES (1, 'Equipos de Aire Acondicionado', 'Unidades condensadoras y
evaporadoras', 1);

INSERT INTO categorias (id, nombre, descripcion, activo)
VALUES (2, 'Gases Refrigerantes', 'Cilindros de gas R-22, R-410A, R-134A', 1);

INSERT INTO categorias (id, nombre, descripcion, activo)
VALUES (3, 'Tuberías y Aislamientos', 'Tuberías de cobre y material aislante', 1);

INSERT INTO productos (id, nombre, descripcion, precio, stock, stock_minimo,
activo, categoria_id)
VALUES (1, 'Aire Acondicionado Split 12000 BTU', 'Inverter, clase A', 1200.00, 15,
3, 1, 1);

INSERT INTO productos (id, nombre, descripcion, precio, stock, stock_minimo,
activo, categoria_id)
VALUES (2, 'Gas Refrigerante R-410A (Boya 11.3kg)', 'Gas ecológico', 180.00, 30,
5, 1, 2);

INSERT INTO productos (id, nombre, descripcion, precio, stock, stock_minimo,
activo, categoria_id)
VALUES (3, 'Tubo de Cobre 1/4 pulgada x 15m', 'Para línea de líquido', 45.50, 100,
10, 1, 3);

-- =====
-- MÓDULO 2: CLIENTES
-- =====
INSERT INTO clientes (id, tipo_cliente, numero_documento, nombres, apellidos,
nombre_comercial, correo, telefono, activo)
VALUES (1, 'PERSONA', '76543210', 'Juan Carlos', 'Perez Quispe', 'Juan Perez',
'juan@gmail.com', '951234567', 1);

INSERT INTO clientes (id, tipo_cliente, numero_documento, razon_social,
nombre_comercial, direccion, correo, telefono, activo)
VALUES (2, 'EMPRESA', '20512345678', 'Clínica San Pablo S.A.', 'Clínica San
Pablo', 'Av. El Polo 789, Surco', 'logistica@sanpablo.pe', '014567890', 1);

-- =====
-- MÓDULO 3: PROVEEDORES
-- =====
INSERT INTO proveedores (id, tipo, nombre_o_razon_social, ruc, direccion,
telefono, correo, contacto_vendedor)
VALUES (1, 'EMPRESA', 'Refrigeración Industrial SAC', '20888888888', 'Av.
Argentina 1500, Lima', '999888777', 'ventas@refriindustrial.pe', 'Carlos Gomez');

INSERT INTO proveedores (id, tipo, nombre_o_razon_social, dni, telefono, correo)
VALUES (2, 'PERSONA_NATURAL', 'Miguel Torres Ramos', '45678912', '987654321',
'miguel.torres@gmail.com');

```

```

-- =====
-- MÓDULO 4: COMPRAS (Cabecera + Detalle)
-- =====
INSERT INTO compras (id, proveedor_id, fecha_compra, numero_factura, moneda,
total)
VALUES (1, 1, SYSDATE - 5, 'F001-0004521', 'PEN', 3600.00);

INSERT INTO detalle_compra (id, compra_id, producto_id, cantidad,
precio_unitario_compra)
VALUES (1, 1, 1, 3, 1200.00);

-- =====
-- MÓDULO 5: COTIZACIONES (Cabecera + Detalle)
-- =====
INSERT INTO cotizaciones (id, fecha, estado, moneda, tipo_cambio, cliente_id,
subtotal, descuento, igv, total, total_soles, total_dolares, observaciones)
VALUES (1, SYSTIMESTAMP, 'ENVIADA', 'PEN', 3.7500, 2, 1016.95, 0.00, 183.05,
1200.00, 1200.00, 320.00, 'Cotización para instalación de 1 equipo split');

INSERT INTO detalle_cotizaciones (id, cotizacion_id, producto_id, cantidad,
precio_unitario, subtotal)
VALUES (1, 1, 1, 1, 1200.00, 1200.00);

-- =====
-- MÓDULO 6: VENTAS (Cabecera + Detalle)
-- =====
INSERT INTO ventas (id, fecha, estado, moneda, tipo_cambio, cliente_id, subtotal,
descuento, igv, total, total_soles, total_dolares)
VALUES (1, SYSTIMESTAMP, 'REGISTRADA', 'PEN', 3.7500, 1, 1016.95, 0.00, 183.05,
1200.00, 1200.00, 320.00);

INSERT INTO detalle_ventas (id, venta_id, producto_id, cantidad, precio_unitario,
subtotal)
VALUES (1, 1, 1, 1, 1200.00, 1200.00);

-- =====
-- MÓDULO 7: PAGOS
-- =====
INSERT INTO pagos (id, fecha_pago, venta_id, moneda, tipo_cambio, monto,
monto_soles, monto_dolares, metodo_pago, observacion)
VALUES (1, SYSTIMESTAMP, 1, 'PEN', 3.7500, 1200.00, 1200.00, 320.00, 'EFECTIVO',
'Pago al contado');

-- =====
-- MÓDULO 8: FACTURACIÓN (Comprobantes Electrónicos)
-- =====
INSERT INTO comprobantes_electronicos (id, tipo_comprobante, serie, correlativo,
fecha_emision, moneda, tipo_cambio, subtotal, descuento, igv, total, total_soles,
total_dolares, estado_sunat, estado_envio_cliente, venta_id)
VALUES (1, 'BOLETA', 'B001', '0000001', SYSTIMESTAMP, 'PEN', 3.7500, 1016.95,
0.00, 183.05, 1200.00, 1200.00, 320.00, 'GENERADO', 'NO_ENVIADO', 1);

-- =====

```

```
-- MÓDULO 9A: POSTVENTA - Instalaciones
-- =====

INSERT INTO instalaciones (id, venta_id, fecha_instalacion, fecha_fin_garantia,
fecha_proximo_mantenimiento, direccion_instalacion, observaciones)
VALUES (1, 1, SYSDATE + 2, ADD_MONTHS(SYSDATE, 12), ADD_MONTHS(SYSDATE, 6), 'Jr.
Los Pinos 456, Juliaca', 'Instalación en segundo piso');

-- =====

-- MÓDULO 9B: POSTVENTA - Mantenimientos
-- =====

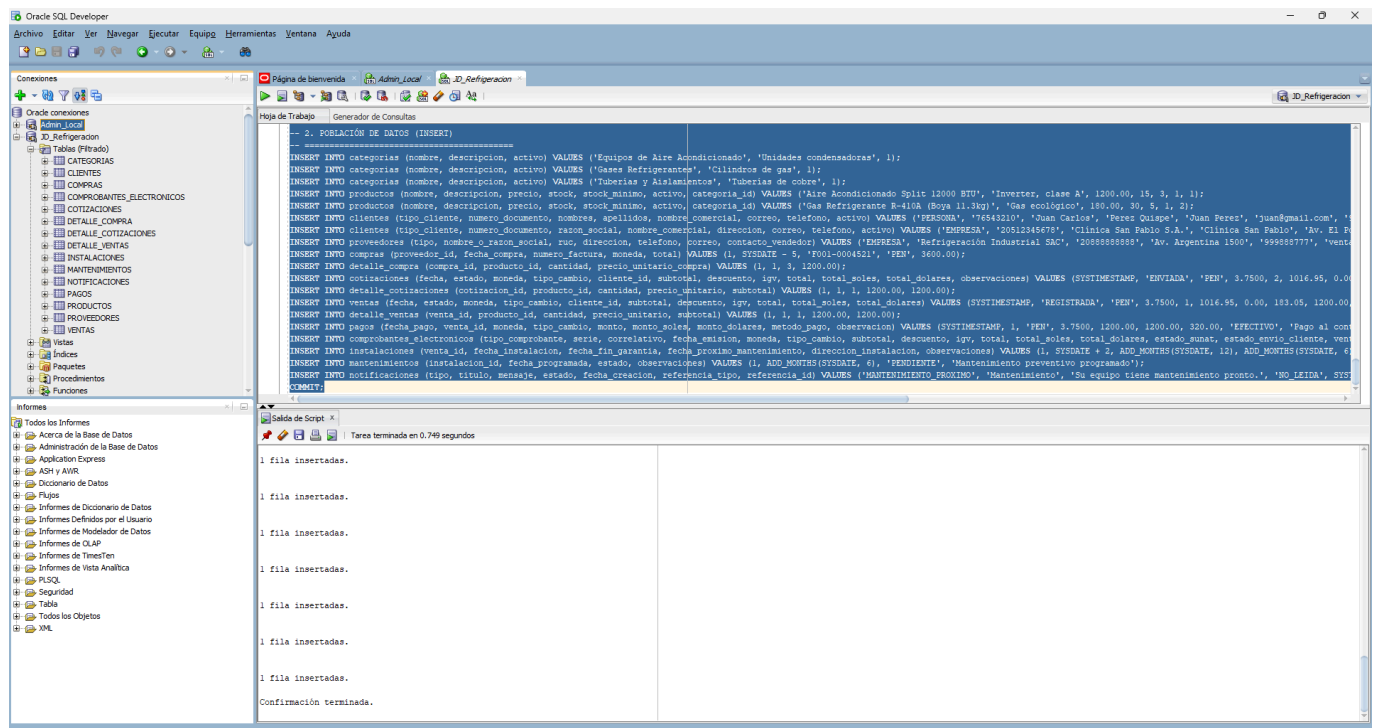
INSERT INTO mantenimientos (id, instalacion_id, fecha_programada, estado,
observaciones)
VALUES (1, 1, ADD_MONTHS(SYSDATE, 6), 'PENDIENTE', 'Primer mantenimiento
preventivo programado');

-- =====

-- MÓDULO 10: NOTIFICACIONES (Transversal)
-- =====

INSERT INTO notificaciones (id, tipo, titulo, mensaje, estado, fecha_creacion,
referencia_tipo, referencia_id)
VALUES (1, 'MANTENIMIENTO_PROXIMO', 'Mantenimiento programado', 'Su equipo tiene
un mantenimiento preventivo programado para dentro de 6 meses.', 'NO_LEIDA',
SYSTIMESTAMP, 'INSTALACION', 1);

COMMIT;
```



2. Procedimientos Almacenados (Stored Procedures)

2.1. Generar Comprobante Electrónico (Módulo Facturación → Ventas → Clientes)

Genera automáticamente un comprobante (Factura o Boleta) evaluando el tipo de cliente de la venta. Si el cliente es EMPRESA genera FACTURA (serie F001), si es PERSONA genera BOLETA (serie B001).

```

CREATE OR REPLACE PROCEDURE SP_GENERAR_COMPROBANTE(
    p_venta_id IN NUMBER
) AS
    v_tipo_doc      VARCHAR2(20);
    v_serie         VARCHAR2(10);
    v_correlativo   VARCHAR2(20);
    v_tipo_cliente  VARCHAR2(20);
    v_subtotal      NUMBER(12,2);
    v_descuento     NUMBER(12,2);
    v_igv           NUMBER(12,2);
    v_total         NUMBER(12,2);
    v_total_soles   NUMBER(12,2);
    v_total_dolares NUMBER(12,2);
    v_moneda        VARCHAR2(10);
    v_tipo_cambio   NUMBER(12,4);
BEGIN
    -- Obtener datos de la venta y el tipo de cliente
    SELECT c.tipo_cliente, v.subtotal, v.descuento, v.igv, v.total,
           v.total_soles, v.total_dolares, v.moneda, v.tipo_cambio
    INTO v_tipo_cliente, v_subtotal, v_descuento, v_igv, v_total,
         v_total_soles, v_total_dolares, v_moneda, v_tipo_cambio
    FROM ventas v
    JOIN clientes c ON v.cliente_id = c.id
    WHERE v.id = p_venta_id;

    -- Determinar tipo de comprobante según tipo de cliente
    IF v_tipo_cliente = 'EMPRESA' THEN
        v_tipo_doc := 'FACTURA';
        v_serie    := 'F001';
    ELSE
        v_tipo_doc := 'BOLETA';
        v_serie    := 'B001';
    END IF;

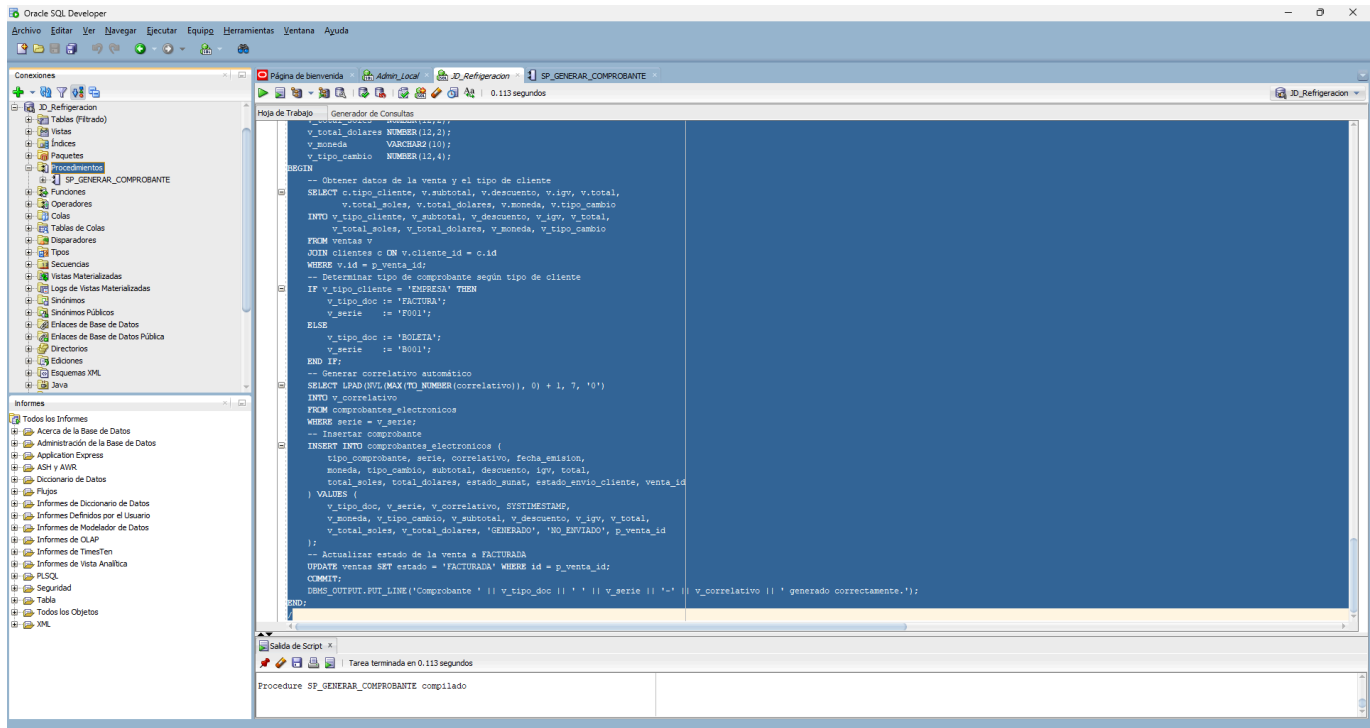
    -- Generar correlativo automático
    SELECT LPAD(NVL(MAX(TO_NUMBER(correlativo)), 0) + 1, 7, '0')
    INTO v_correlativo
    FROM comprobantes_electronicos
    WHERE serie = v_serie;

    -- Insertar comprobante
    INSERT INTO comprobantes_electronicos (
        id, tipo_comprobante, serie, correlativo, fecha_emision,
        moneda, tipo_cambio, subtotal, descuento, igv, total,
        total_soles, total_dolares, estado_sunat, estado_envio_cliente, venta_id
    ) VALUES (
        (SELECT NVL(MAX(id),0)+1 FROM comprobantes_electronicos),
        v_tipo_doc, v_serie, v_correlativo, SYSTIMESTAMP,
        v_moneda, v_tipo_cambio, v_subtotal, v_descuento, v_igv, v_total,
        v_total_soles, v_total_dolares, 'GENERADO', 'NO_ENVIADO', p_venta_id
    );

```

```
-- Actualizar estado de la venta a FACTURADA
UPDATE ventas SET estado = 'FACTURADA' WHERE id = p_venta_id;

COMMIT;
DBMS_OUTPUT.PUT_LINE('Comprobante ' || v_tipo_doc || ' ' || v_serie || '-' ||
v_correlativo || ' generado correctamente.');
```



2.2. Programar Mantenimiento Preventivo (Módulo Postventa)

Registra un nuevo mantenimiento preventivo a partir de una instalación existente. Actualiza automáticamente la fecha del próximo mantenimiento en la tabla de instalaciones.

```
CREATE OR REPLACE PROCEDURE SP_PROGRAMAR_MANTENIMIENTO(
    p_instalacion_id IN NUMBER,
    p_meses_adelante IN NUMBER DEFAULT 6
) AS
    v_fecha_programada DATE;
BEGIN
    -- Calcular fecha programada
    v_fecha_programada := ADD_MONTHS(SYSDATE, p_meses_adelante);

    -- Insertar mantenimiento
    INSERT INTO mantenimientos (id, instalacion_id, fecha_programada, estado,
    observaciones)
    VALUES (
        (SELECT NVL(MAX(id),0)+1 FROM mantenimientos),
        p_instalacion_id,
        v_fecha_programada,
        'PENDIENTE',
```

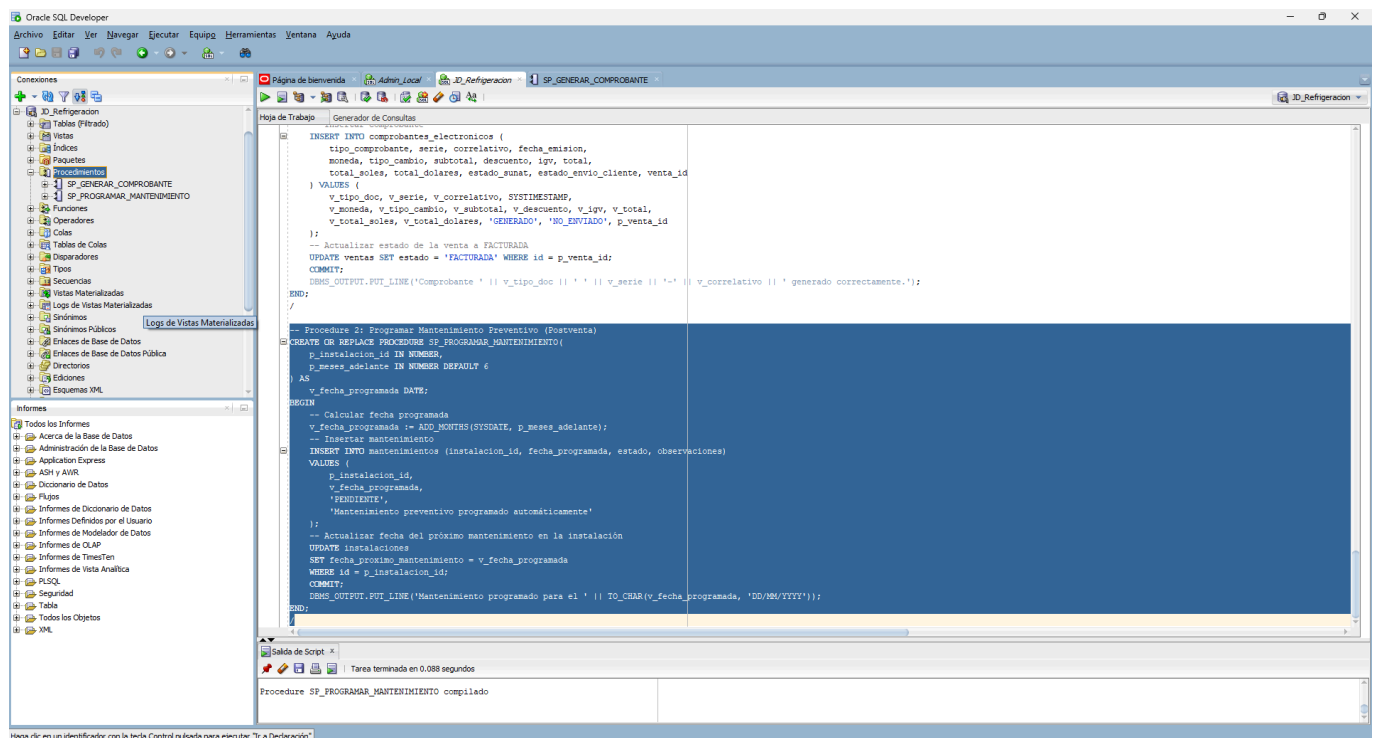
```

        'Mantenimiento preventivo programado automáticamente'
    );

    -- Actualizar fecha del próximo mantenimiento en la instalación
    UPDATE instalaciones
    SET fecha_proximo_mantenimiento = v_fecha_programada
    WHERE id = p_instalacion_id;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Mantenimiento programado para el ' ||
    TO_CHAR(v_fecha_programada, 'DD/MM/YYYY'));
END;
/

```



3. Funciones (Functions)

3.1. Valorización Total del Inventario (Módulo Inventario)

Calcula cuánto capital tiene invertido la empresa en mercadería activa actualmente en almacén (Stock × Precio de cada producto).

```

CREATE OR REPLACE FUNCTION FN_VALORIZACION_INVENTARIO
RETURN NUMBER IS
    v_valorizacion NUMBER(12,2);
BEGIN
    SELECT NVL(SUM(stock * precio), 0)
    INTO v_valorizacion
    FROM productos
    WHERE activo = 1;

    RETURN v_valorizacion;

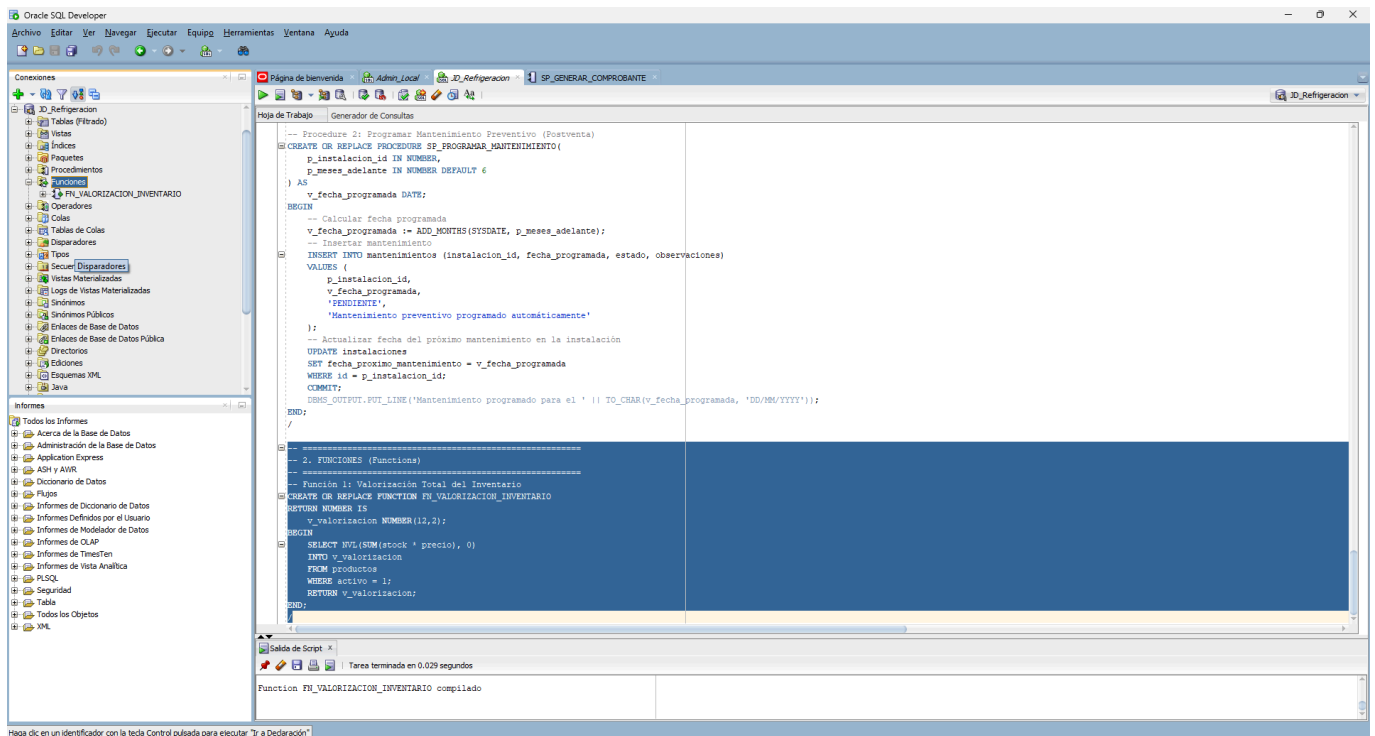
```

```

END;
/

-- Ejemplo de uso:
-- SELECT FN_VALORIZACION_INVENTARIO() FROM DUAL;

```



3.2. Total Cotizado Pendiente por Cliente (Módulo Cotizaciones → Clientes)

Retorna el monto total en soles de todas las cotizaciones que un cliente aún tiene pendientes de aprobación (estado = 'ENVIADA').

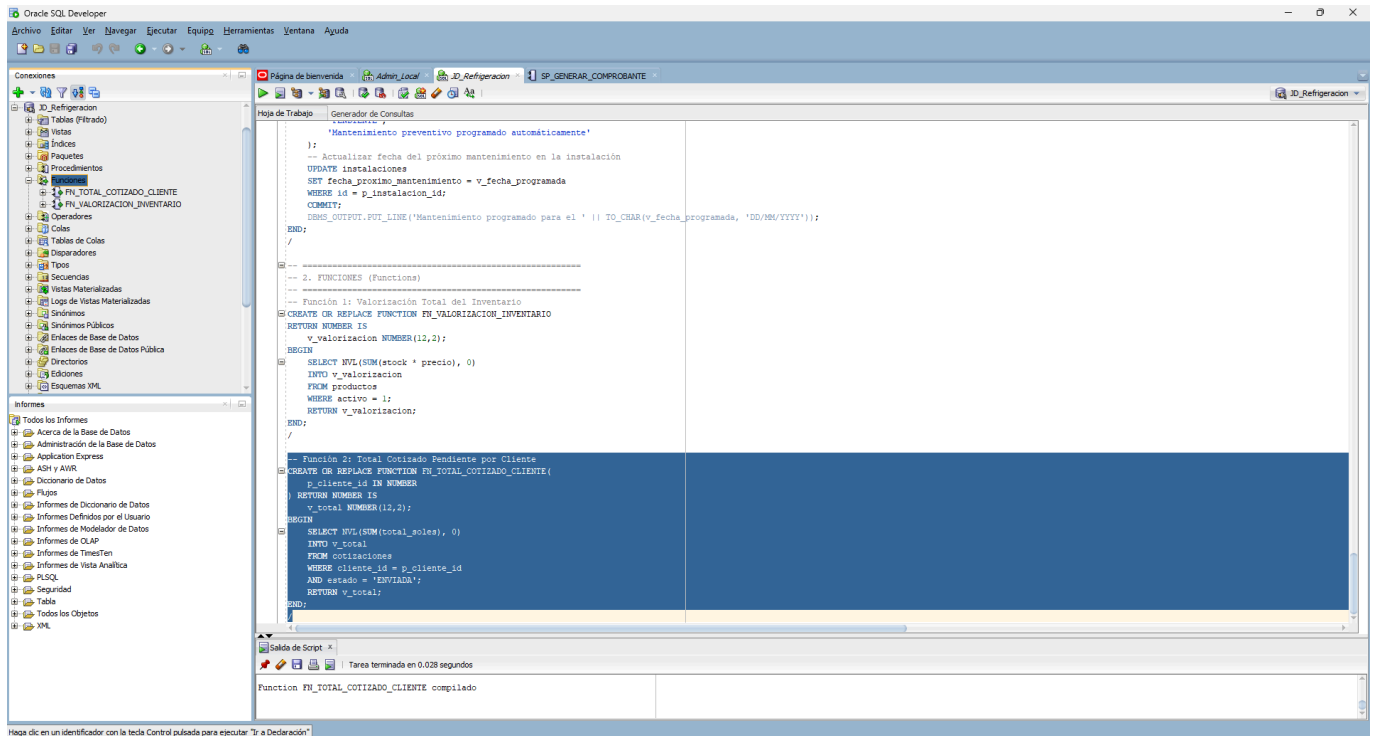
```

CREATE OR REPLACE FUNCTION FN_TOTAL_COTIZADO_CLIENTE(
  p_cliente_id IN NUMBER
) RETURN NUMBER IS
  v_total NUMBER(12,2);
BEGIN
  SELECT NVL(SUM(total_soles), 0)
  INTO v_total
  FROM cotizaciones
  WHERE cliente_id = p_cliente_id
  AND estado = 'ENVIADA';

  RETURN v_total;
END;
/

-- Ejemplo de uso:
-- SELECT FN_TOTAL_COTIZADO_CLIENTE(2) FROM DUAL;

```



4. Disparadores (Triggers)

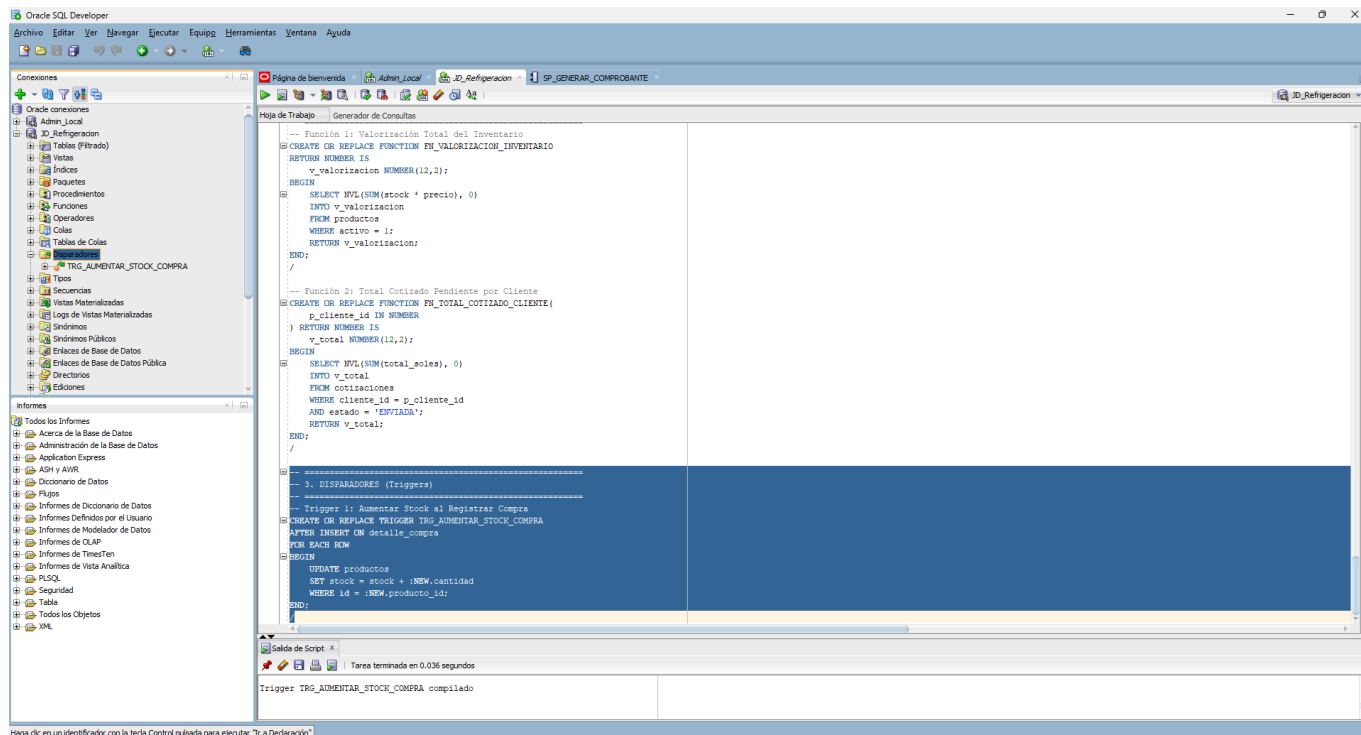
4.1. Aumentar Stock al Registrar Compra (Módulo Compras → Inventario)

Trigger que se ejecuta automáticamente después de insertar un registro en **DETALLE_COMPRA**. Suma la cantidad comprada al stock del producto correspondiente en el inventario.

```

CREATE OR REPLACE TRIGGER TRG_AUMENTAR_STOCK_COMPRA
AFTER INSERT ON detalle_compra
FOR EACH ROW
BEGIN
    UPDATE productos
    SET stock = stock + :NEW.cantidad
    WHERE id = :NEW.producto_id;
END;
/

```



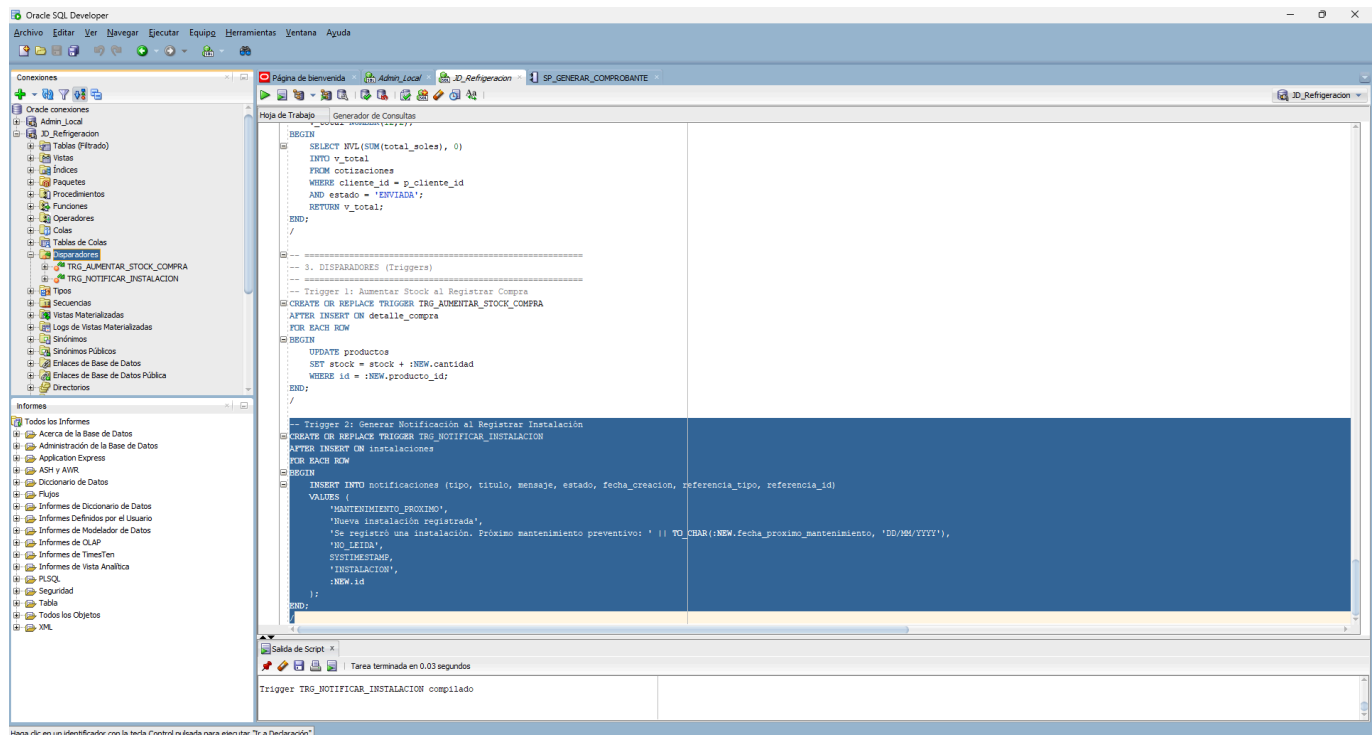
4.2. Generar Notificación al Registrar Instalación (Módulo Postventa → Notificaciones)

Trigger que se ejecuta automáticamente después de insertar una instalación. Crea una notificación informando que se ha programado un mantenimiento preventivo futuro.

```

CREATE OR REPLACE TRIGGER TRG_NOTIFICAR_INSTALACION
AFTER INSERT ON instalaciones
FOR EACH ROW
BEGIN
    INSERT INTO notificaciones (id, tipo, titulo, mensaje, estado, fecha_creacion,
    referencia_tipo, referencia_id)
    VALUES (
        (SELECT NVL(MAX(id),0)+1 FROM notificaciones),
        'MANTENIMIENTO_PROXIMO',
        'Nueva instalación registrada',
        'Se registró una instalación. Próximo mantenimiento preventivo: ' ||
        TO_CHAR(:NEW.fecha_proximo_mantenimiento, 'DD/MM/YYYY'),
        'NO_LEIDA',
        SYSTIMESTAMP,
        'INSTALACION',
        :NEW.id
    );
END;
/

```



Resumen de Relaciones (Foreign Keys)

Módulo Inventario y Compras

PRODUCTOS → ATEGORIAS
 COMPRAS → PROVEEDORES
 DETALLE_COMPRA → COMPRAS
 DETALLE_COMPRA → PRODUCTOS

Módulo Ventas

VENTAS → CLIENTES
 DETALLE_VENTAS → VENTAS
 DETALLE_VENTAS → PRODUCTOS
 PAGOS → VENTAS

Módulo Cotizaciones

COTIZACIONES → CLIENTES
 DETALLE_COTIZACIONES → COTIZACIONES
 DETALLE_COTIZACIONES → PRODUCTOS

Módulo Facturación y Postventa

COMPROBANTES_ELECTRONICOS → VENTAS
 INSTALACIONES → VENTAS
 MANTENIMIENTOS → INSTALACIONES

Modelado en sql power architect

